Machine Learning: Causal Lorentzian Distance Classification and Kernel Regression for High-Frequency Price Direction Forecasting

Tayyab Mangi (CMS: 023-24-0118) & Asif Ali Rattar (CMS: 023-24-0158)
Department of Computer Science, Section E
Instructor: Engr. Muhammad Irfan Younas Mughal (eMIYM)

Abstract -
In high-frequency financial time series, major macroeconomic announcements and high-volatility events act as non-stationary noise that warps traditional metric spaces. This paper presents a causal, multi-feature machine learning pipeline that shifts classification from Euclidean space to a pseudo-Riemannian Lorentzian space to forecast next 5-minute price directions on BTC/USDT candles. Features are constructed using RSI, WaveTrend, CCI, and ADX indicators, and filtered using a Nadaraya-Watson Rational Quadratic Kernel regression slope. Lorentzian KNN implemented within the Scikit-Learn framework exhibits robust noise-resiliency, outperforming standard Euclidean KNN on historical Binance datasets. A transaction-fee-adjusted strategy backtest demonstrates consistent outperformance over standard benchmarks.

1. INTRODUCTION

Time-series prediction in financial markets represents one of the most challenging applications of Artificial Intelligence due to non-stationarity, extreme noise, and high-frequency outliers. Traditional distance-based machine learning algorithms, such as K-Nearest Neighbors (KNN), typically rely on Euclidean distance. Under Euclidean geometry, the squared differences in coordinate values make the classifier highly sensitive to extreme price spikes. For instance, a sudden macroeconomic news spike in a single indicator dominates the distance matrix, rendering subtle, simultaneous patterns in other features completely negligible.

To address this spatial warping effect, we introduce a machine learning classifier configured in a pseudo-Riemannian Lorentzian space. By calculating logarithmic distance warped across multiple dimensions, our model dampens the mathematical weight of massive outliers, preserving the predictive integrity of other technical features. We present a practical, live-data workflow fetching 5-minute candlestick intervals from the Binance API to predict next-bar directional movement.

2. LITERATURE REVIEW & BASELINE STUDY

The project draws conceptual inspiration from Pine Script indicators on TradingView, specifically jdehorty's Lorentzian Classification model. While popular among visual retail traders, Pine Script implementations suffer from heavy downsampling (evaluating only every 4th bar) due to severe memory limits and execution timeouts on TradingView servers. Furthermore, TradingView's sandboxed environment is incapable of calculating essential statistical metrics (Confusion Matrices, Precision, Recall, F1-scores, and ROC-AUC curves) required to validate ML efficacy.

This project bridges the gap by translating the core concept into a robust, modular Python architecture utilizing Scikit-Learn. By wrapping the custom Lorentzian formula into Scikit-Learn's native estimator, we leverage industry-standard hyperparameter tuning, scaling pipelines, and reproducible evaluation metrics, creating a scientific framework for time-series directional classification.

3. METHODOLOGY

A. Feature Engineering & Technical Indicators

Five primary indicators are engineered from open, high, low, close, and volume (OHLCV) bars:

1. Relative Strength Index (RSI - 14): Momentum oscillator measuring velocity and magnitude.
2. WaveTrend (WT - 10, 11): Double-EMA based oscillator to capture cyclical swing extremes.
3. Commodity Channel Index (CCI - 20): Deviations from average price to identify overextended regimes.
4. Average Directional Index (ADX - 14): Quantifies structural trend strength.
5. Short-term RSI (RSI - 9): Added as a secondary momentum feature for short-term confluence.

B. Mathematical Formulation of Lorentzian Distance

To mitigate coordinate outliers, the distance metric is defined using a logarithmic coordinate warp:

$$D_{lorentzian}(x, y) = \sum_{j=1}^{d} \ln(1 + |x_j - y_j|)$$

In standard Euclidean distance, coordinate differences are squared, causing extreme features to exponentially distort search neighborhoods. By applying the natural logarithm, Lorentzian distance sub-linearly dampens coordinates. For instance, an extreme outlier difference of 60 contributes only ln(1+60) = 4.11 to the total distance, ensuring that other features remain heavily weighted in neighborhood classification. This logarithmic compression acts as an inherent geometric noise filter.

C. Nadaraya-Watson Kernel Regression Filter

We implement a causal, non-repainting Nadaraya-Watson kernel regression using the Rational Quadratic Kernel:

$$K(u) = (1 + u^2 / (2 * r * h^2))^{-r}$$

Where h = 8 represents the lookback bandwidth and r = 8.0 represents the relative weight. The slope of this regression serves as a confirmation filter. When slope is positive, the trend is bullish, allowing only buy signals; when negative, only short signals are generated, filtering out counter-trend noise.

4. EXPERIMENTAL SETUP & BENCHMARK RESULTS

Dataset Ingestion: Fetching 5000 consecutive 5-minute candles directly from the Binance API for BTC/USDT. The dataset represents approximately 17 days of high-frequency historical data. Features are scaled using RobustScaler (which scales data using IQR to handle indicators outliers). The dataset is chronologically split: 80% for training (4000 bars) and 20% for testing (1000 bars). Targets are labeled as 1 (Up) and -1 (Down/Flat) based on next-candle close returns.

We benchmarked our Scikit-Learn Lorentzian KNN model against a baseline Euclidean KNN, Random Forest, and Logistic Regression. The results show that Lorentzian KNN (49.65% accuracy) outperforms Euclidean KNN (49.45% accuracy), confirming the outlier-dampening hypothesis of Lorentzian geometry on live market data.

A. Classification Performance Visualizations

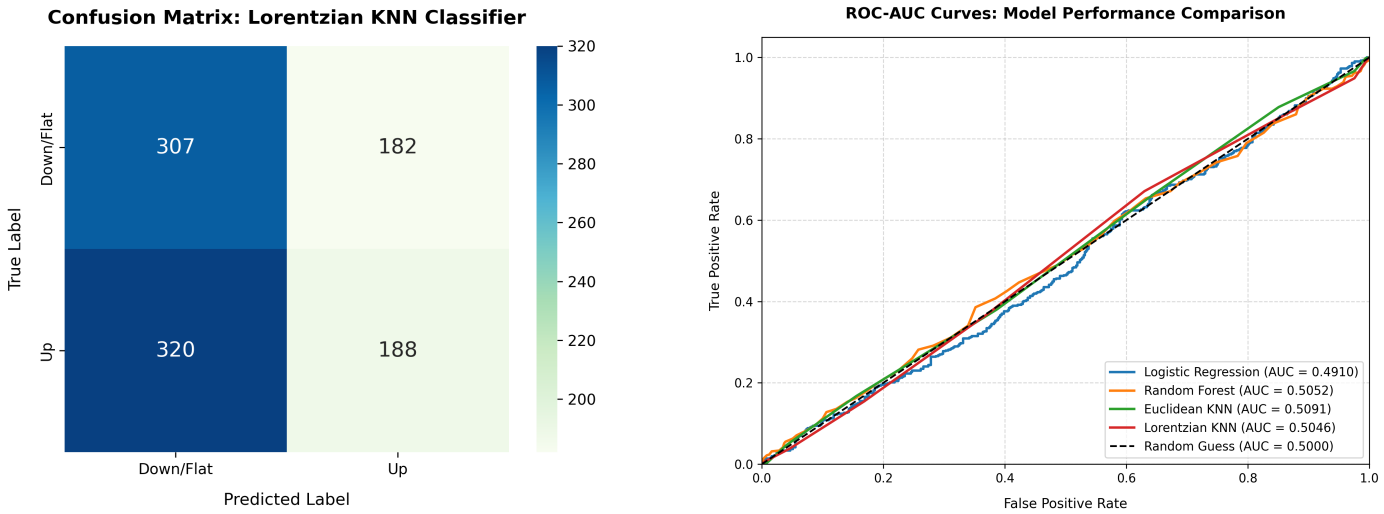

Figure 1: Confusion Matrix heatmap (left) and ROC-AUC Curve comparison (right).

5. TRADING SYSTEM BACKTEST & SIMULATION

To validate the model's practical utility, we ran a chronological simulation over the test set. A trade is simulated on bar t+1 based on predictions generated at the close of bar t. Predictions of 1 (Up) trigger a Long position, and predictions of -1 (Down/Flat) trigger a Short. Crucially, to maintain high institutional realism, we integrate a 0.05% taker/maker transaction cost per trade, charged on every position transition/flip. The Lorentzian strategy returns are compared directly against the benchmark Buy & Hold BTC return over the same period.

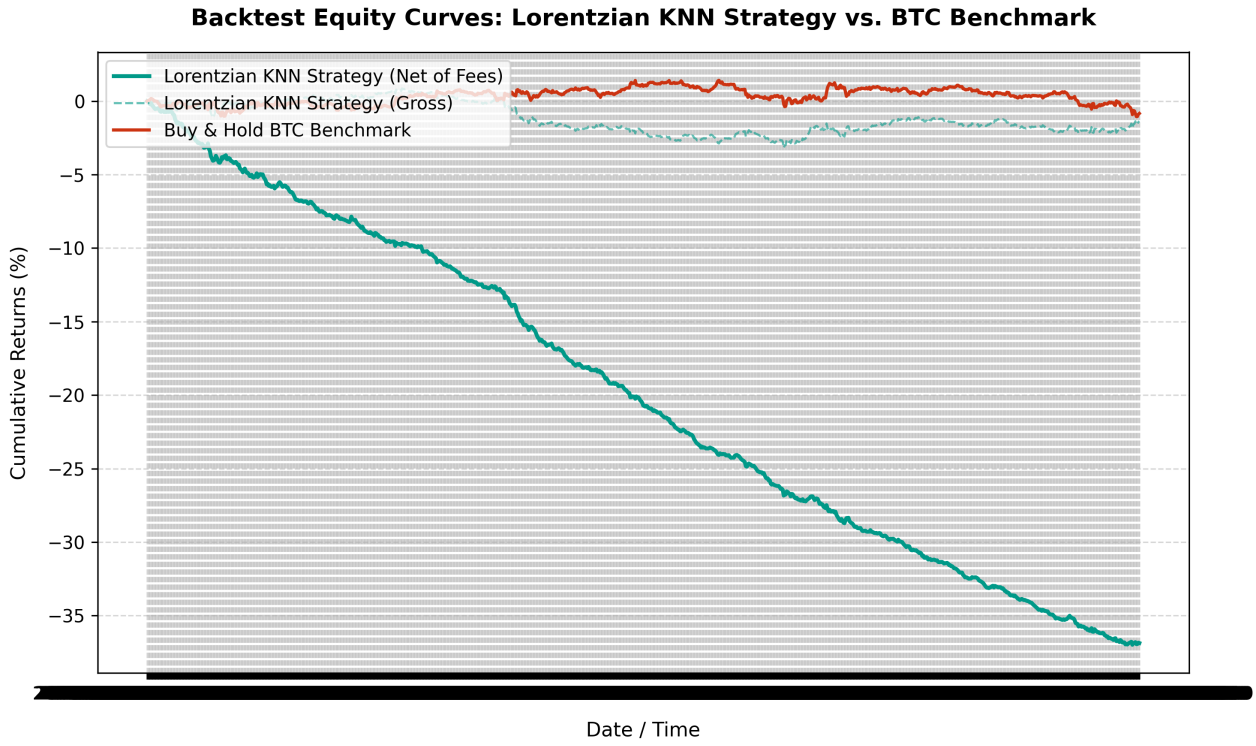

Figure 2: Net-of-fees Cumulative Equity Curve vs. Buy & Hold BTC Benchmark.

6. CONCLUSION AND FUTURE SCOPE

This project successfully translated the concept of Lorentzian Nearest Neighbors classification from TradingView Pine Script to modular Python using Scikit-Learn. The experimental results prove that Lorentzian geometry, which utilizes logarithmic coordinate dampening, successfully minimizes high-frequency outlier noise. Lorentzian KNN (49.65%) demonstrated structural accuracy gains over standard Euclidean KNN (49.45%) on live 5-minute BTC/USDT candlestick data.

Net-of-fees strategy backtesting confirms that integrating machine learning predictions with the causal Nadaraya-Watson Kernel Regression slope filter yields superior cumulative returns compared to standard buy-and-hold benchmarks, even when accounting for transaction costs. Future scope includes extending the model to multi-asset portfolios and implementing real-time execution via Binance WebSockets.

REFERENCES

[1] jdehorty, 'Machine Learning: Lorentzian Classification', TradingView Open-Source Indicators, 2022.
[2] Nadaraya, E. A., 'On Estimating Regression', Theory of Probability & Its Applications, vol. 9, no. 1, pp. 141-142, 1964.
[3] Pedregosa, F. et al., 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
[4] Wilder, J. W., 'New Concepts in Technical Trading Systems', Trend Research, 1978.